Announcements

- Project Phase 2 grades are posted
- Demo submission, commit each TODO
- Demo grading issues, communicate with your section TA and course instructor

Web Engineering & Development (SWE 363)

Interactive Front-End Development

In today's lecture:

- Getting started with React
- JSX
- Components

Reference:

- Zybook: 5.5 to 5.7

5.5 Getting Started with React

What is React?

React is a **JavaScript library** for building user interfaces

- Created by **Facebook (Meta)** in 2013
- Makes building web apps **easier** and **more organized**
- Focuses on **components** - reusable pieces of UI

Think of it as: A better way to build interactive websites!

Why Do We Need React?

Traditional Web Development Problems:

- **Complex DOM manipulation** with vanilla JavaScript
- **Hard to maintain** large applications
- **Repetitive code** for similar UI elements
- **Difficult to manage** state and data flow

Why Do We Need React?

React Solution:

- **Component-based** architecture
- **Automatic UI updates** when data changes
- **Reusable components** reduce code duplication
- **Better organization** and maintainability

React.js: The Documentary

How A Small Team of Developers Created React at Facebook

React vs Traditional Approach

Traditional Way:

```
<!-- HTML -->
<div id="user-card">
  <h3 id="user-name">Loading...</h3>
  <p id="user-email">Loading...</p>
</div>
```

```
// JavaScript - Manual DOM manipulation
document.getElementById('user-name').textContent = 'Ahmed';
document.getElementById('user-email').textContent = 'ahmed@kfupm.edu.sa';
```

React vs Traditional Approach

React Way:

```
// One component handles everything
function UserCard({ name, email }) {
  return (
    <div>
      <h3>{name}</h3>
      <p>{email}</p>
    </div>
  );
}
```

Key Benefits of React

Benefit	Explanation
Reusable Components	Build once, use everywhere. Like LEGO blocks for web development
Virtual DOM	Faster updates than direct DOM manipulation. React figures out what changed and updates only that part
Strong Ecosystem	Backed by Meta (Facebook). Huge community and many tools
Single Page Applications (SPA)	No page reloads. Smooth, app-like experience

How React Works

The React Process:

1. **You write components** using JSX
2. **React creates a Virtual DOM** (in-memory representation)
3. **When data changes**, React compares old vs new Virtual DOM
4. **React updates only the changed parts** in the real DOM

Result:

Faster performance **Less work for developers** **Automatic updates**

Setting Up a React Project

Using Vite (Modern & Fast):

```
# Create new React project
npm create vite@latest my-react-app


# Navigate to project
cd my-react-app

# Install dependencies
npm install

# Start development server
npm run dev
```

React Project Structure 📂

```
my-react-app/
├── node_modules/          # Dependencies
├── public/                # Static files (images, etc.)
├── src/                   # Your code lives here
│   ├── App.jsx            # Main app component
│   ├── main.jsx           # Entry point
│   └── components/        # Your custom components
├── package.json           # Project configuration
└── vite.config.js         # Vite settings
```

How React Renders Your App

1. `main.jsx` (Entry Point):

```jsx
import React from 'react'
import ReactDOM from 'react-dom/client'
import App from './App.jsx'

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
)
```

How React Renders Your App

2. `App.jsx` (Main Component):

```
function App() {
  return (
    <div>
      <h1>Hello React!</h1>
    </div>
  )
}

export default App
```

Your First React Component

Create `src/components/Welcome.jsx`:

```jsx
function Welcome() {
  return (
    <div>
      <h1>Welcome to React!</h1>
      <p>This is my first component</p>
    </div>
  );
}

export default Welcome;
```

Using Your Component

Use it in `App.jsx`:

```jsx
import Welcome from './components/Welcome';

function App() {
  return (
    <div>
      <Welcome />
    </div>
  );
}

export default App;
```

5.6 JSX

What is JSX?

JSX = JavaScript XML

- A special syntax that lets you write **HTML-like code inside JavaScript**
- Makes React components **easier to read and write**
- **Not HTML** - it's JavaScript that looks like HTML

Example:

```
function MyComponent() {
  return <h1>Hello, World!</h1>;
}
```

Why Use JSX?

Without JSX (Traditional JavaScript):

```
function createElement() {
  const h1 = document.createElement('h1');
  h1.textContent = 'Hello, World!';
  h1.className = 'title';
  return h1;
}
```

With JSX (React):

```
function MyComponent() {
  return <h1 className="title">Hello, World!</h1>;
}
```

JSX is much cleaner and easier to read!

JSX Rules 📋

1. Must Return a Single Element

```
// WRONG - Multiple elements
function BadComponent() {
  return ( <h1>Title</h1>
           <p>Paragraph</p> );
}
// CORRECT - Wrapped in one element
function GoodComponent() {
  return (
    <div>
      <h1>Title</h1>
      <p>Paragraph</p>
    </div>
  );
}
```

JSX Rules

2. Use `className` Instead of `class`

```
// WRONG
<div class="container">Content</div>
// CORRECT
<div className="container">Content</div>
```

3. Self-Closing Tags Must Have `/`

```
//  WRONG
<img src="image.jpg">
//  CORRECT
<img src="image.jpg" />
```

JSX Rules (cont.) 📋

4. Use `{}` for JavaScript Expressions

```
function Greeting({ name, age }) {
  return (
    <div>
      <h1>Hello, {name}!</h1>
      <p>You are {age} years old</p>
      <p>Next year you'll be {age + 1}</p>
    </div>
  );
}
```

JSX Rules

5. Use `camelCase` for Attributes

```
//  WRONG
<div tabindex="0" onclick={handleClick}>

// CORRECT
<div tabIndex="0" onClick={handleClick}>
```

JSX Examples 💡

Basic JSX:

```jsx
function UserCard() {
  const userName = "Ahmed";
  const userEmail = "ahmed@kfupm.edu.sa";

  return (
    <div className="card">
      <h2>{userName}</h2>
      <p>{userEmail}</p>
      <button onClick={() => alert('Hello!')}>
        Click Me
      </button>
    </div>
  );
}
```

JSX with Conditional Rendering

```jsx
function UserStatus({ isLoggedIn, userName }) {
  return (
    <div>
      {isLoggedIn ? (
        <p>Welcome back, {userName}!</p>
      ) : (
        <p>Please log in</p>
      )}
    </div>
  );
}
```

JSX with Lists

```
function StudentList({ students }) {
  return (
    <ul>
      {students.map(student => (
        <li key={student.id}>
          {student.name} - {student.grade}
        </li>
      ))}
    </ul>
  );
}
```

5.7 Components

What is a Component?

A **component** is a **reusable piece of UI**

- Like a **function** that returns HTML
- **Small, focused** on one thing
- **Reusable** - use it multiple times
- **Composable** - combine components to build complex UIs

Think of it as: A custom HTML element you create!

Component Analogy

Building a House:

- **Foundation** (App component)
- **Rooms** (Header, Sidebar, MainContent components)
- **Furniture** (Button, Card, Input components)

Building a Website:

- **App** (main container)
- **Header, Navigation, Footer** (layout components)
- **Button, Card, Form** (reusable components)

Creating and Using Components

Step 1: Create `src/components/StudentCard.jsx`:

```jsx
function StudentCard() {
  return (
    <div className="student-card">
      <h3>Ahmed Al-Saeed</h3>
      <p>ID: 202333345</p>
      <p>Department: Computer Science</p>
    </div>
  );
}

export default StudentCard;
```

Creating and Using Components

Step 2: Use it in `App.jsx`:

```jsx
import StudentCard from './components/StudentCard';

function App() {
  return (
    <div>
      <h1>Student Directory</h1>
      <StudentCard />
    </div>
  );
}
```

Making Components Dynamic with Props

Updated `StudentCard.jsx`:

```jsx
function StudentCard({ name, id, department }) {
  return (
    <div className="student-card">
      <h3>{name}</h3>
      <p>ID: {id}</p>
      <p>Department: {department}</p>
    </div>
  );
}

export default StudentCard;
```

Using Props in App.jsx

Using it in `App.jsx`:

```
import StudentCard from './components/StudentCard';

function App() {
  return (
    <div>
      <h1>Student Directory</h1>
      <StudentCard
        name="Ahmed Al-Saud" id="202312345" department="Computer Science"
      />
      <StudentCard
        name="Sara Al-Rashid" id="202312346" department="Software Engineering"
      />
    </div>
);}
```

Props Explained

Props = Properties

- **Data passed from parent to child** component
- Make components **reusable** and **dynamic**
- **Read-only** - child can't change props
- Like **function parameters** but for components

Component Best Practices

1. One Component Per File

```
src/
├── components/
│   ├── StudentCard.jsx
│   ├── Header.jsx
│   └── Footer.jsx
```

2. Descriptive Names

```
// Good
function UserProfileCard() { ... }
// Bad
function Card() { ... }
```

Component Best Practices

3. Export Default

```
function MyComponent() { ... }
export default MyComponent;
```

4. Keep Components Small

```
// Good - One responsibility
function StudentName({ name }) { return <h3>{name}</h3>; }
// Bad - Too many responsibilities
function StudentCard({ name, id, department, grades, schedule }) {
  // Too much code here
}
```

Component Best Practices

5. Use Meaningful Props

```
// Good
<StudentCard name="Ahmed" studentId="123" />

// Bad
<StudentCard a="Ahmed" b="123" />
```

30m

Demo

5.2 React Starter

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
```

Next Class

- More on React